HW #4.3: Reproducible Data Analysis

Hojin Yoon

2026-04-13

Busiest Airports Analysis

Table Table 1 summarizes annual passenger traffic from 2020 through 2025 for six major international airports: Atlanta (ATL), Frankfurt (FRA), Seoul Incheon (ICN), John F. Kennedy (JFK), London Heathrow (LHR), and Beijing Daxing (PKX). The table allows the reader to compare exact passenger counts across airports and years, while Figure Figure 1 highlights overall trends and changes over time.

Several clear patterns emerge from these visualizations. Atlanta (ATL) consistently had the highest passenger traffic across all years, increasing from approximately 42.9 million passengers in 2020 to over 106 million in 2025. Heathrow (LHR) also experienced strong growth, rising from about 22 million passengers in 2020 to over 84 million by 2025.

Some airports show sharper fluctuations. Seoul Incheon (ICN) experienced a major drop from 2020 to 2021 before recovering significantly in later years. Similarly, Beijing Daxing (PKX) dropped in 2022 before rebounding. These differences indicate that recovery patterns were not uniform across airports.

Overall, Table Table 1 provides precise numerical comparisons, while Figure Figure 1 makes it easier to see the shape of recovery and long-term trends. Together, they show that while all airports experienced growth after early declines, ATL remained dominant and ICN and PKX showed the most dramatic recoveries.

Table 1: Passenger traffic for six busy airports from 2020 to 2025.

Airport	2020	2021	2022	2023	2024	2025
ATL	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
FRA	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
ICN	12,094,851	3,198,909	17,869,759	56,235,412	71,212,515	74,071,475
JFK	16,630,642	30,788,322	55,287,693	62,464,331	63,265,984	62,629,455
LHR	22,111,469	19,393,145	61,614,508	79,183,364	83,884,572	84,463,000
PKX	16,091,449	25,051,012	10,277,623	39,410,776	49,441,029	53,618,949

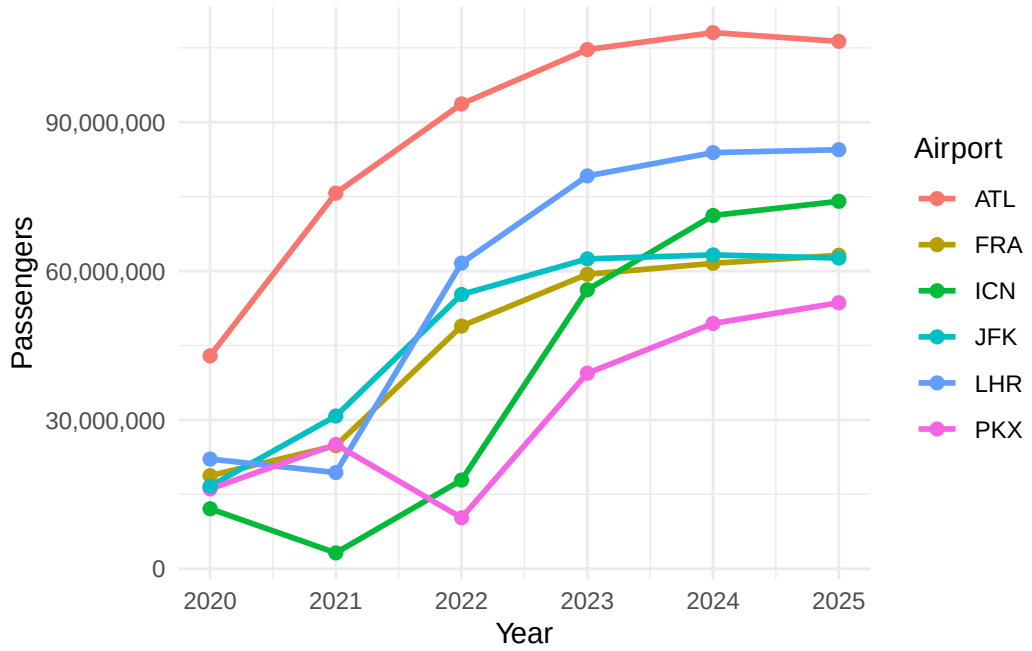


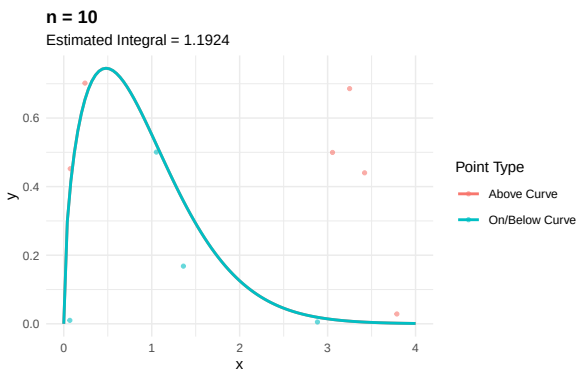
Figure 1: Passenger traffic trends for six busy airports from 2020 to 2025.

Monte Carlo Numerical Integration

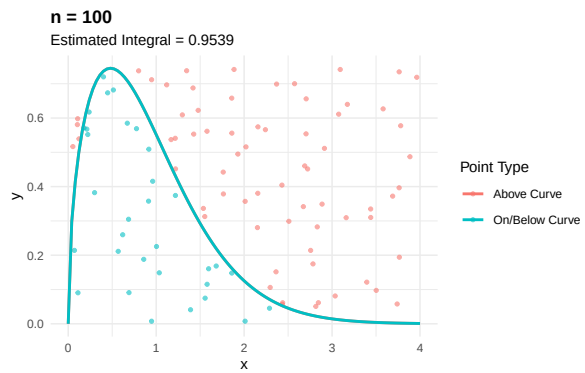
Monte Carlo numerical integration estimates the area under a curve by randomly generating points inside a rectangle that fully contains the function. For each simulated point, the x-coordinate is used to evaluate the function, and the y-coordinate is compared to that value. If the point falls on or below the curve, it contributes to the estimated proportion of the rectangle's area that lies under the function. Multiplying that proportion by the area of the rectangle gives an estimate of the integral.

For this analysis, I used the Weibull density function with shape parameter 1.5 and scale parameter 1 over the interval from 0 to 4. Since the interval from 0 to 4 captures essentially all of the visible density, I expect the value of the numerical integral over this interval to be very close to 1.

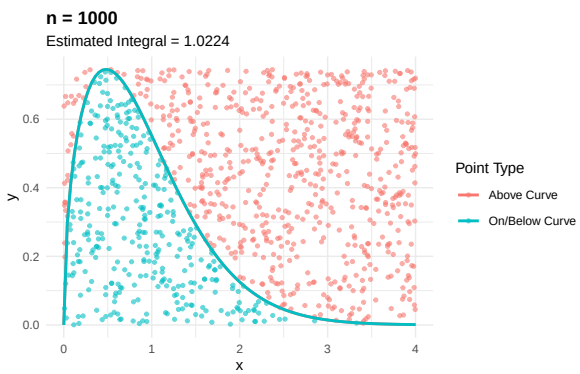
Figure Figure 2 shows four Monte Carlo simulations using different numbers of random points. As the number of points increases from 10 to 10,000, the estimate becomes more stable and gets closer to 1. With only 10 points, the estimate is much less reliable because random variation has a much larger effect. By 1,000 and 10,000 points, the estimated values are both very close to 1, which supports the conclusion that the integral is approximately 1. The visual pattern also becomes more convincing at higher sample sizes because the points fill the rectangle more evenly, making the proportion under the curve easier to judge.



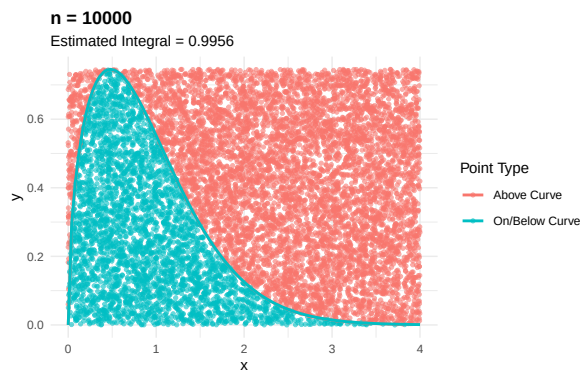
(a) Simulation with n = 10.



(b) Simulation with n = 100.



(c) Simulation with n = 1000.



(d) Simulation with n = 10000.

Figure 2: Monte Carlo numerical integration of the Weibull density function using four different simulation sizes.

Planning and Prompting GenAI Tools

Plan

Goal

The goal is to tidy the calcium dataset and create a visualization that clearly shows how ulnar calcium levels change over time for two groups: the null treatment group and the diet/exercise treatment group.

Needs

- Convert the dataset from wide format to tidy format
- Correctly identify and label treatment groups
- Represent time as a variable rather than separate columns
- Ensure each row represents one participant at one time point
- Create a visualization that compares trends over time between groups
  Steps

1. Load the dataset using `read_csv()`
2. Recognize that the first four columns correspond to the null group and the next four correspond to the diet/exercise group
3. Separate the dataset into two groups and assign a treatment label to each
4. Add a subject ID variable to track individuals
5. Combine the groups into one dataset
6. Use `pivot_longer()` to convert the data into tidy format
7. Convert the time variable into an ordered factor
8. Create a line plot showing calcium levels over time
9. Color by treatment group and optionally include group-level summary trends
10. Evaluate whether the visualization clearly communicates differences between groups

---

GenAI Output (Plan-Informed Prompt)

Below is the response generated using ChatGPT based on a structured, plan-informed prompt.

```r
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v forcats   1.0.0     v stringr   1.5.2
v lubridate 1.9.5     v tibble    3.3.0
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x readr::col_factor() masks scales::col_factor()
x purrr::discard()    masks scales::discard()
x dplyr::filter()     masks stats::filter()
x dplyr::lag()        masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

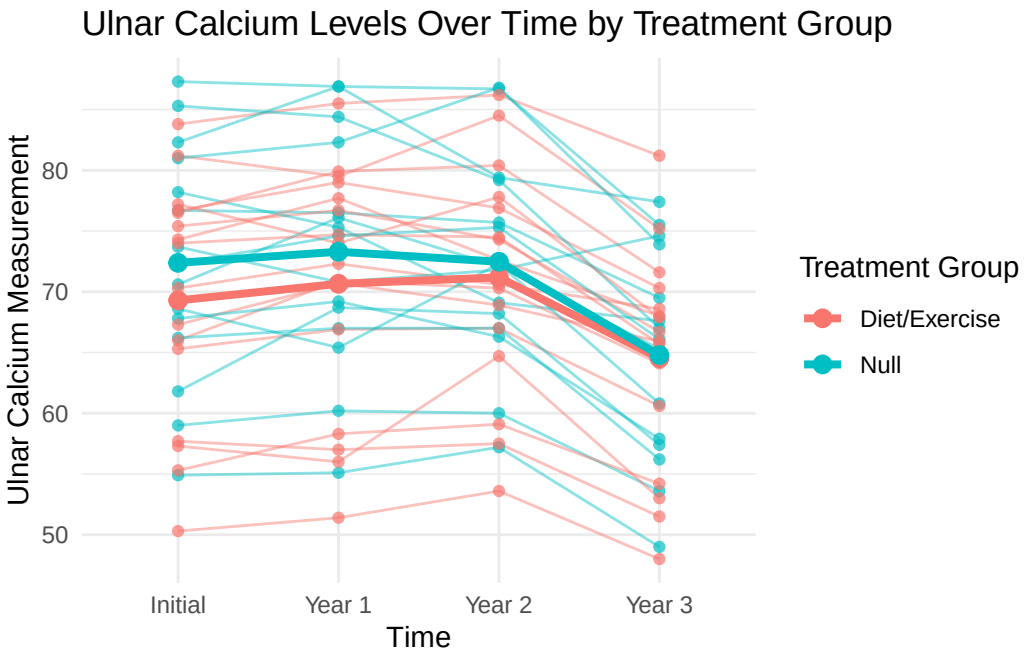
```r
calcium <- read_csv("calcium.csv")
```

```
New names:
Rows: 16 Columns: 8
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(8): y0...1, y1...2, y2...3, y3...4, y0...5, y1...6, y2...7, y3...8
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `y0` -> `y0...1`
* `y1` -> `y1...2`
* `y2` -> `y2...3`
* `y3` -> `y3...4`
* `y0` -> `y0...5`
* `y1` -> `y1...6`
* `y2` -> `y2...7`
* `y3` -> `y3...8`
```

```r
null_group <- calcium %>%
  select(1:4) %>%
  rename(
    Initial = 1,
    `Year 1` = 2,
    `Year 2` = 3,
    `Year 3` = 4
  ) %>%
  mutate(
    subject_id = row_number(),
    treatment = "Null"
  )

diet_exercise_group <- calcium %>%
  select(5:8) %>%
  rename(
    Initial = 1,
    `Year 1` = 2,
    `Year 2` = 3,
    `Year 3` = 4
  ) %>%
  mutate(
    subject_id = row_number(),
    treatment = "Diet/Exercise"
  )

calcium_tidy <- bind_rows(null_group, diet_exercise_group) %>%
  pivot_longer(
    cols = c(Initial, `Year 1`, `Year 2`, `Year 3`),
    names_to = "time",
    values_to = "calcium"
  ) %>%
  mutate(
    time = factor(time, levels = c("Initial", "Year 1", "Year 2", "Year 3"))
  )

ggplot(
  calcium_tidy,
  aes(
    x = time,
    y = calcium,
    group = interaction(treatment, subject_id),
```

```r
    color = treatment
  )
) +
  geom_line(alpha = 0.45) +
  geom_point(alpha = 0.7) +
  stat_summary(
    aes(group = treatment),
    fun = mean,
    geom = "line",
    linewidth = 1.4
  ) +
  stat_summary(
    aes(group = treatment),
    fun = mean,
    geom = "point",
    size = 2.8
  ) +
  labs(
    title = "Ulnar Calcium Levels Over Time by Treatment Group",
    x = "Time",
    y = "Ulnar Calcium Measurement",
    color = "Treatment Group"
  ) +
  theme_minimal()
```

```
Warning: Removed 4 rows containing non-finite outside the scale range
(`stat_summary()`).
Removed 4 rows containing non-finite outside the scale range
(`stat_summary()`).

Warning: Removed 4 rows containing missing values or values outside the scale range
(`geom_line()`).

Warning: Removed 4 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Explanation

The generated response first reads in the dataset and separates it into two groups based on the column structure. A subject ID and treatment label are added to ensure that each participant can be tracked across time. The data is then combined and reshaped into tidy format so that each row represents one participant at one time point.

The resulting visualization uses line plots to show individual trajectories over time, along with summary lines representing the average trend for each treatment group. This makes it possible to compare how calcium levels change across time and between groups.

Generic Prompt Output

Below is the result generated using a generic prompt.

Explanation

The generic prompt produced a visualization that displays the average value of each column in the dataset. Each point represents the mean of a column (such as y0, y1, y2, etc.), allowing for a quick comparison of overall values.

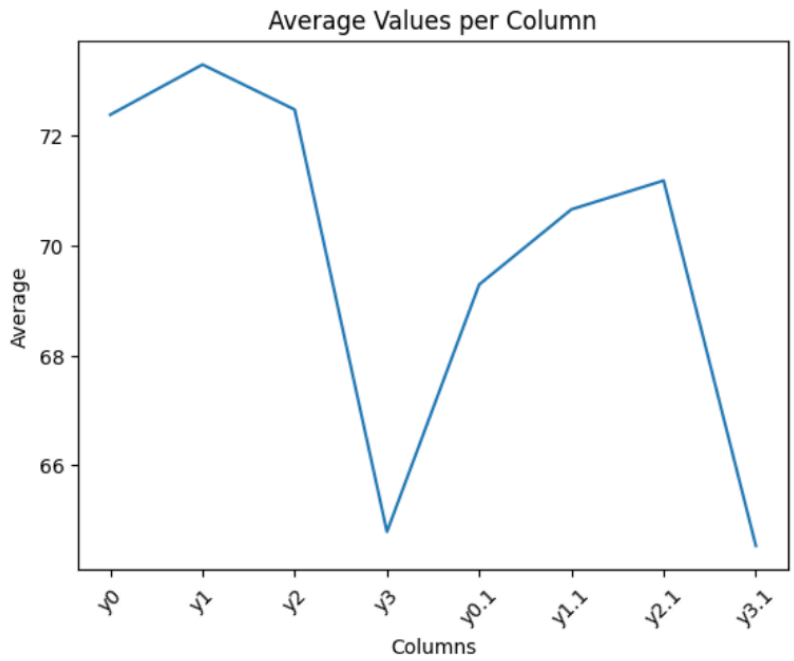


Figure 3: Generic GenAI output showing average values for each column in the calcium dataset.

From this plot, we can see that most values fall within a similar range, with some columns having slightly higher or lower averages. However, the visualization does not reflect the structure of the dataset, such as the progression over time or the presence of two different treatment groups.

As a result, while the plot provides a basic summary of the data, it does not capture the key relationships needed to understand how calcium levels change over time or differ between groups.

Comparison

The plan-informed prompt produced a much more meaningful and accurate result compared to the generic prompt. Because the structured prompt clearly explained how the dataset was organized, the generated output correctly separated the two treatment groups and preserved the longitudinal structure of the data. It also included subject identifiers, allowing individual trends over time to be visualized.

In contrast, the generic prompt produced a plot that only shows the average value of each column. While this provides a basic summary, it ignores the underlying structure of the dataset.

Specifically, it does not distinguish between the null treatment group and the diet/exercise group, and it does not treat time as a variable. Instead, it treats each column as an independent measurement.

The difference in visualization is important. The plan-informed output uses line plots to show how calcium levels change over time for each participant, along with overall group trends. This directly supports the goal of comparing changes over time between treatment groups. The generic output, however, removes all time-based and individual-level information by averaging the data, making it much less useful for understanding the context.

Overall, this comparison demonstrates that providing a clear and detailed prompt leads to significantly better results when using GenAI tools. The structured prompt ensures that the data is interpreted correctly and that the visualization aligns with the goals of the analysis, while the generic prompt results in a less informative summary.

Reflection

Throughout this course, I developed a much stronger understanding of how to work with data in a structured and meaningful way. At the beginning of the semester, I mainly focused on just getting correct outputs, but now I understand the importance of organizing data, creating clear visualizations, and explaining results in a way that others can understand.

One of the biggest things I learned was how to work with tidy data. Earlier in the course, I did not fully understand why data needed to be reshaped, but assignments like the calcium dataset showed me how important it is for each row to represent a single observation. Using tools like `pivot_longer()` helped me transform messy data into a format that made analysis and visualization much easier.

Another important concept I learned was how to choose appropriate visualizations. In the busiest airports analysis, I learned how line graphs can effectively show trends over time, while in the Monte Carlo section I learned how visualizing random points at different sample sizes can demonstrate how estimates become more stable. This helped me understand that the choice of visualization directly affects how well the data communicates a story.

I also learned how Monte Carlo numerical integration works conceptually and visually. Before this, I only thought of integration as a mathematical process, but this assignment helped me see how random sampling can be used to approximate an area. By comparing simulations with different numbers of points, I could clearly see how increasing sample size improves accuracy, which made the concept much more intuitive.

Additionally, I gained experience using Quarto to create reproducible data analysis documents. Learning how to separate narrative text from code, label figures and tables, and use cross-references made my work more organized and professional. This was very different from simply writing code in a script, because it required me to think about both the analysis and how to present it.

Finally, this assignment helped me understand how to use GenAI tools more effectively. I saw that a detailed and structured prompt leads to much better results than a vague one. When I used a plan-informed prompt, the output correctly interpreted the dataset and produced a meaningful visualization. In contrast, a generic prompt led to a less useful result. This showed me that GenAI is most effective when combined with clear thinking and planning, rather than relying on it without guidance.

Overall, I feel that I have improved not only my technical skills in R and data wrangling, but also my ability to think critically about data and communicate insights clearly.

GenAI Usage Appendix

Usage 1: Planning and Prompting Section

Tool Used: ChatGPT (OpenAI GPT-5.3)
Date: April 13, 2026

Prompt: I have a dataset called calcium.csv from a longitudinal study of 31 women over four years. The dataset is currently not in tidy format.

The first four columns correspond to the null treatment group (Initial, Year 1, Year 2, Year 3), and the next four columns correspond to the diet/exercise treatment group (Initial, Year 1, Year 2, Year 3).

I want to: 1. Tidy and restructure this dataset so that each row represents one observation (one participant at one time point) 2. Clearly distinguish between the two treatment groups 3. Create a visualization that compares how calcium levels change over time between the two groups

Please: - Write R code using tidyverse to clean and reshape the data - Add a subject ID variable - Convert the data into long/tidy format - Create a line plot showing calcium levels over time - Color by treatment group - Optionally include a summary trend (mean line) for each group - Include clear comments explaining each step

Response: library(tidyverse)

calcium <- read_csv("calcium.csv")

null_group <- calcium %>% select(1:4) %>% rename(Initial = 1, `Year 1` = 2, `Year 2` = 3, `Year 3` = 4) %>% mutate(subject_id = row_number(), treatment = "Null")

diet_exercise_group <- calcium %>% select(5:8) %>% rename(Initial = 1, `Year 1` = 2, `Year 2` = 3, `Year 3` = 4) %>% mutate(subject_id = row_number(), treatment = "Diet/Exercise")

calcium_tidy <- bind_rows(null_group, diet_exercise_group) %>% pivot_longer(cols = c(Initial, `Year 1`, `Year 2`, `Year 3`), names_to = "time", values_to = "calcium") %>% mutate(time = factor(time, levels = c("Initial", "Year 1", "Year 2", "Year 3")))

ggplot(calcium_tidy, aes(x = time, y = calcium, group = interaction(treatment, subject_id), color = treatment)) + geom_line(alpha = 0.45) + geom_point(alpha = 0.7) + stat_summary(aes(group = treatment), fun = mean, geom = "line", linewidth = 1.4) + stat_summary(aes(group = treatment), fun = mean, geom = "point", size = 2.8) + labs(title = "Ulnar Calcium Levels Over Time by Treatment Group", x = "Time", y = "Ulnar Calcium Measurement", color = "Treatment Group") + theme_minimal()

Usage 2: Generic Prompt

Tool Used: ChatGPT (OpenAI GPT-5.3)
Date: April 13, 2026

Prompt: Create a data visualization for these data. Calcium.csv

Response: I created a visualization showing the average value of each column in your dataset.

What this shows: - Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.). - You can quickly compare which variables tend to have higher or lower values overall. - For example, some columns cluster around ~70–73, while others drop closer to ~65.

If you want something more insightful, I can also create: - Boxplots (to show spread and outliers) - Time series plots (if rows represent time/order) - Correlation heatmaps - Side-by-side comparisons (e.g., y0 vs y0.1)

Usage 3: Version Information Check

Tool Used: ChatGPT (OpenAI GPT-5.3)
Date: March 31, 2026

Prompt: What is your version information?

Response: You're chatting with ChatGPT based on the GPT-5.3 architecture.

Key details: - Model: GPT-5.3 - Knowledge cutoff: August 2025 - Current date (system): March 31, 2026

Code Appendix

```r
# ==========================================================
# Setup
# Load packages used throughout the document
# ==========================================================

library(dplyr)
library(tidyr)
library(knitr)
library(ggplot2)
library(scales)
library(readr)

# ==========================================================
# Busiest Airports Analysis
# Create the dataset used for the table and line plot
# ==========================================================

busiest_airports_tidy <- data.frame(
  airport = c(
    rep("Hartsfield-Jackson Atlanta International Airport", 6),
    rep("Frankfurt Airport", 6),
    rep("Beijing Daxing International Airport", 6),
    rep("Heathrow Airport", 6),
    rep("Seoul Incheon International Airport", 6),
    rep("John F. Kennedy International Airport", 6)
  ),
  airport_short = c(
    rep("ATL", 6),
    rep("FRA", 6),
    rep("PKX", 6),
    rep("LHR", 6),
    rep("ICN", 6),
    rep("JFK", 6)
  ),
  year = rep(2020:2025, 6),
  passengers = c(
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    18768601, 24812849, 48918482, 59355389, 61564957, 63189666,
    16091449, 25051012, 10277623, 39410776, 49441029, 53618949,
    22111469, 19393145, 61614508, 79183364, 83884572, 84463000,
```

```r
    12094851, 3198909, 17869759, 56235412, 71212515, 74071475,
    16630642, 30788322, 55287693, 62464331, 63265984, 62629455
  )
)

# Create the formatted table of airport passenger counts
busiest_airports_tidy %>%
  select(airport_short, year, passengers) %>%
  pivot_wider(names_from = year, values_from = passengers) %>%
  arrange(airport_short) %>%
  rename(Airport = airport_short) %>%
  mutate(across(`2020`:`2025`, ~ format(.x, big.mark = ","))) %>%
  kable(align = "c")

# Create the line plot of passenger trends over time
ggplot(
  busiest_airports_tidy,
  aes(x = year, y = passengers, color = airport_short, group = airport_short)
) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = label_comma()) +
  labs(
    x = "Year",
    y = "Passengers",
    color = "Airport"
  ) +
  theme_minimal()

# ==========================================================
# Monte Carlo Numerical Integration
# Define a function to generate random points in a rectangle
# ==========================================================

monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {

  if (!is.numeric(n) || length(n) != 1 || n <= 0 || n %% 1 != 0) {
    stop("n must be one positive whole number.")
  }

  if (!is.numeric(x_min) || !is.numeric(x_max) ||
```

```r
      length(x_min) != 1 || length(x_max) != 1) {
    stop("x_min and x_max must each be one numeric value.")
  }

  if (x_min >= x_max) {
    stop("x_min must be less than x_max.")
  }

  if (!is.numeric(y_min) || !is.numeric(y_max) ||
      length(y_min) != 1 || length(y_max) != 1) {
    stop("y_min and y_max must each be one numeric value.")
  }

  if (y_min >= y_max) {
    stop("y_min must be less than y_max.")
  }

  x <- runif(n = n, min = x_min, max = x_max)
  y <- runif(n = n, min = y_min, max = y_max)

  data.frame(
    point_id = 1:n,
    x = x,
    y = y
  )
}

# Create a plotting function for the Monte Carlo simulations
make_mc_plot <- function(n_points) {

  x_min <- 0
  x_max <- 4
  y_min <- 0
  y_max <- max(
    dweibull(seq(x_min, x_max, length.out = 10000), shape = 1.5, scale = 1)
  )

  mc_data <- monte_carlo_points(
    n = n_points,
    x_min = x_min,
    x_max = x_max,
    y_min = y_min,
```

```r
    y_max = y_max
  )

  mc_data_labeled <- mc_data %>%
    mutate(
      f_x = dweibull(x, shape = 1.5, scale = 1),
      point_type = if_else(y <= f_x, "On/Below Curve", "Above Curve")
    )

  rectangle_area <- (x_max - x_min) * (y_max - y_min)

  mc_estimate <- mc_data_labeled %>%
    summarize(
      proportion_under = mean(point_type == "On/Below Curve"),
      estimate = proportion_under * rectangle_area
    ) %>%
    pull(estimate)

  ggplot(mc_data_labeled, aes(x = x, y = y, color = point_type)) +
    geom_point(alpha = 0.6, size = 1) +
    stat_function(
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      xlim = c(x_min, x_max),
      linewidth = 0.9
    ) +
    scale_x_continuous(limits = c(x_min, x_max)) +
    scale_y_continuous(limits = c(y_min, y_max)) +
    labs(
      title = paste("n =", n_points),
      subtitle = paste("Estimated Integral =", round(mc_estimate, 4)),
      x = "x",
      y = "y",
      color = "Point Type"
    ) +
    theme_minimal(base_size = 10) +
    theme(
      legend.position = "right",
      plot.title = element_text(face = "bold")
    )
}
```

```r
# Generate the four Monte Carlo plots used in the document
make_mc_plot(10)
make_mc_plot(100)
make_mc_plot(1000)
make_mc_plot(10000)

# ==========================================================
# Planning and Prompting GenAI Tools
# Tidy the calcium dataset and create a plan-informed plot
# ==========================================================

calcium <- read_csv("calcium.csv")

null_group <- calcium %>%
  select(1:4) %>%
  rename(
    Initial = 1,
    `Year 1` = 2,
    `Year 2` = 3,
    `Year 3` = 4
  ) %>%
  mutate(
    subject_id = row_number(),
    treatment = "Null"
  )

diet_exercise_group <- calcium %>%
  select(5:8) %>%
  rename(
    Initial = 1,
    `Year 1` = 2,
    `Year 2` = 3,
    `Year 3` = 4
  ) %>%
  mutate(
    subject_id = row_number(),
    treatment = "Diet/Exercise"
  )

calcium_tidy <- bind_rows(null_group, diet_exercise_group) %>%
  pivot_longer(
    cols = c(Initial, `Year 1`, `Year 2`, `Year 3`),
```

```r
    names_to = "time",
    values_to = "calcium"
  ) %>%
  mutate(
    time = factor(time, levels = c("Initial", "Year 1", "Year 2", "Year 3"))
  )

# Plot the plan-informed output for the calcium data
ggplot(
  calcium_tidy,
  aes(
    x = time,
    y = calcium,
    group = interaction(treatment, subject_id),
    color = treatment
  )
) +
  geom_line(alpha = 0.45) +
  geom_point(alpha = 0.7) +
  stat_summary(
    aes(group = treatment),
    fun = mean,
    geom = "line",
    linewidth = 1.4
  ) +
  stat_summary(
    aes(group = treatment),
    fun = mean,
    geom = "point",
    size = 2.8
  ) +
  labs(
    title = "Ulnar Calcium Levels Over Time by Treatment Group",
    x = "Time",
    y = "Ulnar Calcium Measurement",
    color = "Treatment Group"
  ) +
  theme_minimal()

# ==========================================================
# Generic Prompt Output
# Create the weaker generic-response version for comparison
```

```r
# ==========================================================

# Calculate column means
col_means <- colMeans(calcium)

# Create a simple plot of column averages
plot(
  col_means,
  type = "b",
  main = "Average Values per Column",
  xlab = "Columns",
  ylab = "Average Calcium Value"
)

# ================================
# End of Code Appendix
# ================================
```